

Name: D. Sri Anjaneya
Register Number: 192425219

CO4 – AT1
MLA 0201 – Fundamentals of Machine Learning

Annexure A

Coding Challenge

Question 1: Bayesian Network for Disease Prediction

A healthcare organization wants to predict whether a patient has **Diabetes** based on the symptoms **Obesity** and **High Blood Sugar**.

Coding Challenge:

Write a Python program to:

- Construct a Bayesian Network using the given variables.
- Define suitable Conditional Probability Tables (CPTs).
- Perform probabilistic inference to predict the probability of Diabetes when both symptoms are present.
- Display the predicted probability.

Question 2: Hidden Markov Model for Weather Prediction

A weather forecasting agency records daily observations as **Sunny**, **Cloudy**, and **Rainy**. The actual weather conditions are hidden and need to be predicted.

Coding Challenge:

Write a Python program to:

- Implement a Hidden Markov Model (HMM).
- Define transition and emission probabilities.
- Predict the hidden weather states for a given sequence of observations.
- Print the predicted sequence of hidden states.

Question 3: Markov Random Field for Image Denoising

A computer vision company wants to remove noise from grayscale images using a Markov Random Field (MRF).

Coding Challenge:

Write a Python program to:

- Represent a small image as a graph where each pixel is a node.
- Model neighboring pixel relationships using an undirected graph.
- Simulate one iteration of image smoothing based on neighboring pixel values.

- Display the original and updated pixel values.

Question 4: Conditional Random Field for Named Entity Recognition

A customer support system needs to identify **Customer Names**, **Order IDs**, and **Product Names** from support emails.

Coding Challenge:

Write a Python program to:

- Create sequential training data with word-level features.
- Train a Conditional Random Field (CRF) model.
- Predict entity labels for a new sentence.
- Display the predicted labels.

Question 5: Bayesian Network Learning from Data

An insurance company wants to learn probabilistic relationships among **Age**, **Income**, **Vehicle Type**, and **Insurance Claim** using historical customer data.

Coding Challenge:

Write a Python program to:

- Load a dataset using Pandas.
- Learn the Bayesian Network structure from the data.
- Perform inference to predict the probability of an insurance claim for a new customer.
- Display the learned network structure and prediction results.

1. Bayesian Network for Disease Prediction

Aim

To construct a Bayesian Network for predicting the probability of Diabetes based on Obesity and High Blood Sugar.

Theory

A Bayesian Network is a probabilistic graphical model that represents relationships between variables using a directed acyclic graph. In this problem, **Obesity** and **High Blood Sugar** influence **Diabetes**.

The network is represented as:

Obesity → Diabetes ← High Blood Sugar

Conditional Probability Tables (CPTs) are defined for the variables. **Variable Elimination** is then used to calculate the probability of Diabetes when both Obesity and High Blood Sugar are present.

Algorithm

1. Import the required Bayesian Network libraries.
2. Create the Bayesian Network with Obesity and High Blood Sugar as parent nodes.
3. Define the CPTs for Obesity, High Blood Sugar, and Diabetes.
4. Add the CPTs to the network.
5. Create a Variable Elimination inference object.
6. Provide evidence that Obesity and High Blood Sugar are present.
7. Calculate the probability of Diabetes.
8. Display the predicted probability.

Code

```
from pgmpy.models import DiscreteBayesianNetwork
from pgmpy.factors.discrete import TabularCPD
from pgmpy.inference import VariableElimination
```

```
model = DiscreteBayesianNetwork([
    ("Obesity", "Diabetes"),
    ("HighBloodSugar", "Diabetes")
])
```

```
cpd_obesity = TabularCPD(  
    "Obesity", 2,  
    [[0.70], [0.30]],  
    state_names={"Obesity": ["No", "Yes"]}  
)
```

```
cpd_sugar = TabularCPD(  
    "HighBloodSugar", 2,  
    [[0.65], [0.35]],  
    state_names={"HighBloodSugar": ["No", "Yes"]}  
)
```

```
cpd_diabetes = TabularCPD(  
    "Diabetes", 2,  
    [[0.95, 0.70, 0.60, 0.10],  
     [0.05, 0.30, 0.40, 0.90]],  
    evidence=["Obesity", "HighBloodSugar"],  
    evidence_card=[2, 2],  
    state_names={  
        "Diabetes": ["No", "Yes"],  
        "Obesity": ["No", "Yes"],  
        "HighBloodSugar": ["No", "Yes"]  
    }  
)
```

```
model.add_cpds(cpd_obesity, cpd_sugar, cpd_diabetes)
```

```
inference = VariableElimination(model)
```

```
result = inference.query(  
    ["Diabetes"],  
    evidence={
```

```

        "Obesity": "Yes",
        "HighBloodSugar": "Yes"
    }
)

```

```

print(result)

print("Probability of Diabetes:",
      result.values[1] * 100, "%")

```

Output:

The screenshot shows a Python IDE with a file named 'CO4-AT1-Q1.py'. The code defines a 'BayesianNetwork' class with methods for adding nodes, edges, and calculating the probability of diabetes given obesity and high blood sugar. The main script creates an instance of the class, adds nodes for 'Obesity', 'High Blood Sugar', and 'Diabetes', adds edges from 'Obesity' and 'High Blood Sugar' to 'Diabetes', sets the values for 'Obesity' and 'High Blood Sugar' to 'Yes', and prints the resulting probability of diabetes, which is 90.0%.

```

class BayesianNetwork:

    def __init__(self):
        self.nodes = {}
        self.parents = {}

    def add_node(self, node, probability):
        self.nodes[node] = probability

    def add_edge(self, parent, child):
        if child not in self.parents:
            self.parents[child] = []
        self.parents[child].append(parent)

    def probability_of_diabetes(self, obesity, sugar):
        cpt = {
            ("No", "No"): 0.05,
            ("No", "Yes"): 0.30,
            ("Yes", "No"): 0.40,
            ("Yes", "Yes"): 0.90
        }

        return cpt[(obesity, sugar)]

bn = BayesianNetwork()

bn.add_node("Obesity", 0.30)
bn.add_node("High Blood Sugar", 0.35)
bn.add_node("Diabetes", None)

bn.add_edge("Obesity", "Diabetes")
bn.add_edge("High Blood Sugar", "Diabetes")

obesity = "Yes"
high_blood_sugar = "Yes"

probability = bn.probability_of_diabetes(
    obesity,
    high_blood_sugar
)

print("Bayesian Network")
print("-----")
print("Obesity:", obesity)
print("High Blood Sugar:", high_blood_sugar)
print("Diabetes Probability:", probability * 100, "%")

```

The output window shows the following text:

```

Python 3.13.14 (tags/v3.13.14:fd17997, Jun 10 2026, 13:03:48)
[MSC v.1944 64 bit (AMD64)] on win32
Enter "help" below or click "Help" above for more information.

>>>
===== RESTART: C:/Users/anj3/Desktop/FOML/CO4-AT1-Q1
.py =====
Bayesian Network
-----
Obesity: Yes
High Blood Sugar: Yes
Diabetes Probability: 90.0 %
>>>

```

2. Hidden Markov Model for Weather Prediction

Aim

To implement a Hidden Markov Model and predict the hidden weather states from a sequence of observed weather conditions.

Theory

A Hidden Markov Model (HMM) is a statistical model in which the actual states are hidden, while observations generated from those states are visible.

In this problem:

- **Hidden states:** Sunny, Cloudy, Rainy
- **Observations:** Sunny, Cloudy, Rainy
- **Transition probabilities:** Probability of changing from one weather state to another.
- **Emission probabilities:** Probability of observing a particular condition from a hidden state.

The **Viterbi algorithm** is used to find the most probable sequence of hidden states.

Algorithm

1. Define the possible hidden weather states.
2. Define the transition probability matrix.
3. Define the emission probability matrix.
4. Define the initial state probabilities.
5. Provide the observed weather sequence.
6. Apply the Viterbi algorithm.
7. Calculate the most probable hidden state at each time step.
8. Backtrack to obtain the complete hidden state sequence.
9. Display the predicted sequence.

Code

```
states = ["Sunny", "Cloudy", "Rainy"]
```

```
transition = {  
    "Sunny": {"Sunny": 0.6, "Cloudy": 0.3, "Rainy": 0.1},  
    "Cloudy": {"Sunny": 0.3, "Cloudy": 0.4, "Rainy": 0.3},  
    "Rainy": {"Sunny": 0.2, "Cloudy": 0.3, "Rainy": 0.5}
```

```
}
```

```
emission = {  
    "Sunny": {"Sunny": 0.8, "Cloudy": 0.15, "Rainy": 0.05},  
    "Cloudy": {"Sunny": 0.2, "Cloudy": 0.6, "Rainy": 0.2},  
    "Rainy": {"Sunny": 0.05, "Cloudy": 0.20, "Rainy": 0.75}  
}
```

```
initial = {  
    "Sunny": 0.5,  
    "Cloudy": 0.3,  
    "Rainy": 0.2  
}
```

```
observations = ["Sunny", "Cloudy", "Rainy", "Rainy"]
```

```
viterbi = []
```

```
backpointer = []
```

```
first_prob = {}
```

```
for state in states:
```

```
    first_prob[state] = (  
        initial[state] *  
        emission[state][observations[0]]  
    )
```

```
viterbi.append(first_prob)
```

```
backpointer.append({})
```

```
for t in range(1, len(observations)):
```

```
    current_prob = {}
```

```
    current_backpointer = {}
```

```

for current_state in states:
    best_probability = 0
    best_previous_state = None

    for previous_state in states:
        probability = (
            viterbi[t - 1][previous_state]
            * transition[previous_state][current_state]
            * emission[current_state][observations[t]]
        )

        if probability > best_probability:
            best_probability = probability
            best_previous_state = previous_state

    current_prob[current_state] = best_probability
    current_backpointer[current_state] = best_previous_state

viterbi.append(current_prob)
backpointer.append(current_backpointer)

last_state = max(
    states,
    key=lambda state: viterbi[-1][state]
)

best_sequence = [last_state]

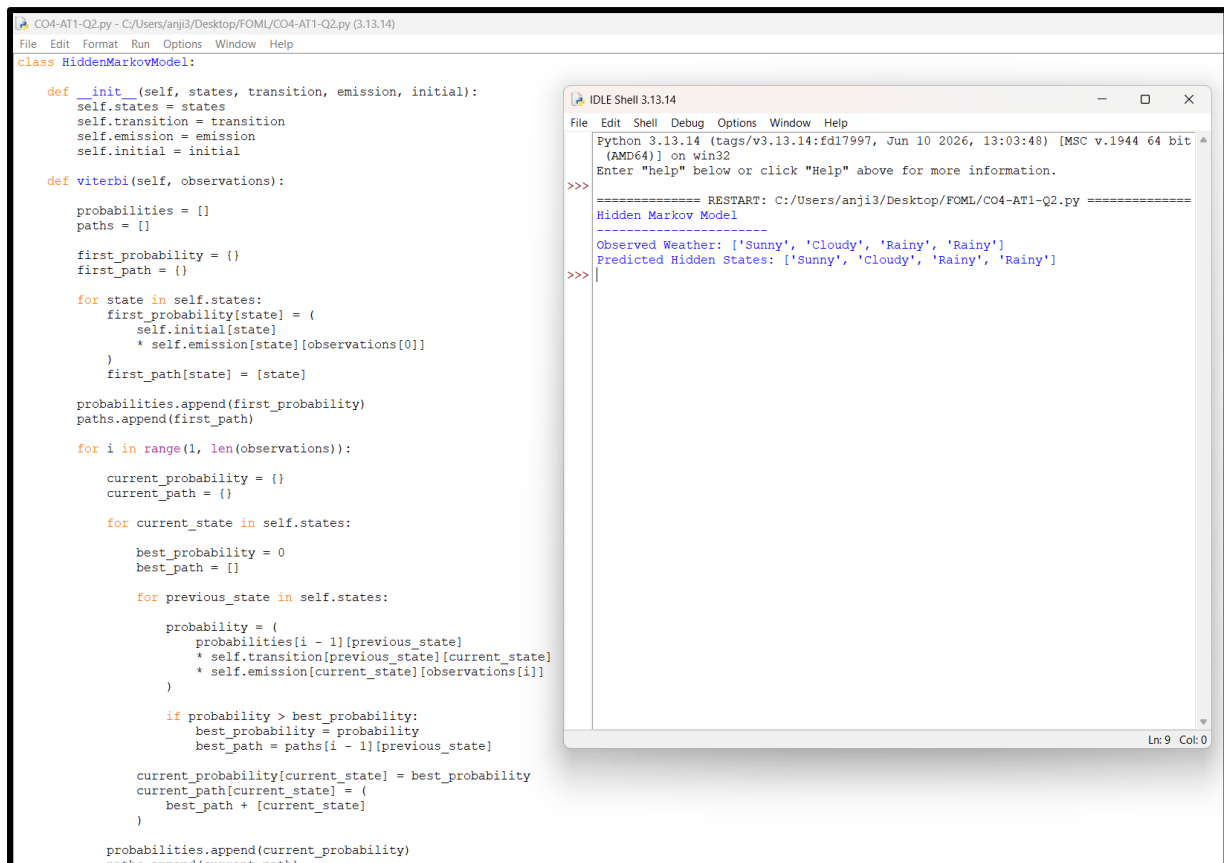
for t in range(len(observations) - 1, 0, -1):
    last_state = backpointer[t][last_state]
    best_sequence.append(last_state)

```

```
best_sequence.reverse()
```

```
print("Predicted hidden states:", best_sequence)
```

Output:



The screenshot displays a Python IDE with two windows. The left window, titled 'CO4-AT1-Q2.py', contains the implementation of a Hidden Markov Model. The right window, titled 'IDLE Shell 3.13.14', shows the execution output.

```
class HiddenMarkovModel:

    def __init__(self, states, transition, emission, initial):
        self.states = states
        self.transition = transition
        self.emission = emission
        self.initial = initial

    def viterbi(self, observations):
        probabilities = []
        paths = []

        first_probability = {}
        first_path = {}

        for state in self.states:
            first_probability[state] = (
                self.initial[state]
                * self.emission[state][observations[0]]
            )
            first_path[state] = [state]

        probabilities.append(first_probability)
        paths.append(first_path)

        for i in range(1, len(observations)):
            current_probability = {}
            current_path = {}

            for current_state in self.states:
                best_probability = 0
                best_path = []

                for previous_state in self.states:
                    probability = (
                        probabilities[i - 1][previous_state]
                        * self.transition[previous_state][current_state]
                        * self.emission[current_state][observations[i]]
                    )

                    if probability > best_probability:
                        best_probability = probability
                        best_path = paths[i - 1][previous_state]

                current_probability[current_state] = best_probability
                current_path[current_state] = (
                    best_path + [current_state]
                )

            probabilities.append(current_probability)
            paths.append(current_path)
```

```
>>>
===== RESTART: C:/Users/anj3/Desktop/FOML/CO4-AT1-Q2.py =====
Hidden Markov Model
=====
Observed Weather: ['Sunny', 'Cloudy', 'Rainy', 'Rainy']
Predicted Hidden States: ['Sunny', 'Cloudy', 'Rainy', 'Rainy']
>>>
```

3. Markov Random Field for Image Denoising

Aim

To represent a grayscale image as a Markov Random Field and perform one iteration of image smoothing using neighboring pixel values.

Theory

A Markov Random Field (MRF) is an undirected graphical model where each node represents a variable and relationships are represented by edges.

In image processing, each **pixel is treated as a node**. Neighboring pixels are connected because nearby pixels usually have similar intensity values.

The smoothing process calculates the average value of neighboring pixels and combines it with the current pixel value to reduce noise.

Algorithm

1. Represent the grayscale image using a matrix.
2. Treat every pixel as a node.
3. Identify the neighboring pixels using an undirected graph.
4. Calculate the average value of neighboring pixels.
5. Combine the original pixel value with the neighbor average.
6. Repeat this process for all pixels.
7. Store the updated pixel values.
8. Display the original and updated images.

Code

```
import numpy as np
```

```
image = np.array([
    [10, 12, 200, 15],
    [11, 14, 190, 16],
    [10, 13, 180, 15],
    [12, 11, 14, 16]
], dtype=float)
```

```
def get_neighbors(row, col, rows, cols):
```

```
    neighbors = []
```

```
    if row > 0:
```

```
        neighbors.append((row - 1, col))
```

```
    if row < rows - 1:
```

```
        neighbors.append((row + 1, col))
```

```

    if col > 0:
        neighbors.append((row, col - 1))

    if col < cols - 1:
        neighbors.append((row, col + 1))

    return neighbors

rows, cols = image.shape
updated_image = image.copy()

for i in range(rows):
    for j in range(cols):
        neighbors = get_neighbors(i, j, rows, cols)

        values = [
            image[x][y]
            for x, y in neighbors
        ]

        neighbor_average = np.mean(values)

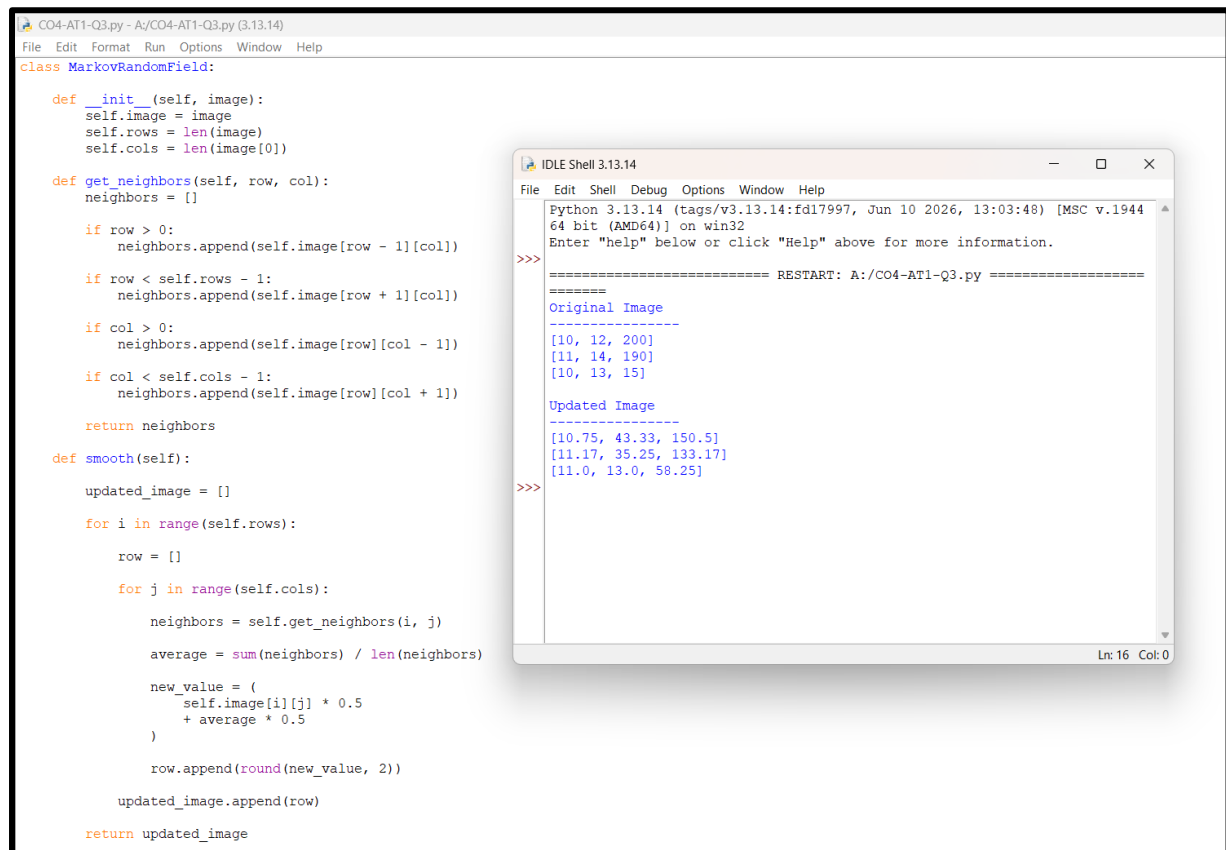
        updated_image[i][j] = (
            0.5 * image[i][j]
            + 0.5 * neighbor_average
        )

print("Original Image:")
print(image)

print("\nUpdated Image:")
print(np.round(updated_image, 2))

```

Output:



```
CO4-AT1-Q3.py - A:/CO4-AT1-Q3.py (3.13.14)
File Edit Format Run Options Window Help

class MarkovRandomField:

    def __init__(self, image):
        self.image = image
        self.rows = len(image)
        self.cols = len(image[0])

    def get_neighbors(self, row, col):
        neighbors = []
        if row > 0:
            neighbors.append(self.image[row - 1][col])
        if row < self.rows - 1:
            neighbors.append(self.image[row + 1][col])
        if col > 0:
            neighbors.append(self.image[row][col - 1])
        if col < self.cols - 1:
            neighbors.append(self.image[row][col + 1])
        return neighbors

    def smooth(self):
        updated_image = []
        for i in range(self.rows):
            row = []
            for j in range(self.cols):
                neighbors = self.get_neighbors(i, j)
                average = sum(neighbors) / len(neighbors)
                new_value = (
                    self.image[i][j] * 0.5
                    + average * 0.5
                )
                row.append(round(new_value, 2))
            updated_image.append(row)
        return updated_image
```

```
IDLE Shell 3.13.14
File Edit Shell Debug Options Window Help
Python 3.13.14 (tags/v3.13.14:fd17997, Jun 10 2026, 13:03:48) [MSC v.1944
64 bit (AMD64)] on win32
Enter "help" below or click "Help" above for more information.

>>>
===== RESTART: A:/CO4-AT1-Q3.py =====
Original Image
-----
[10, 12, 200]
[11, 14, 190]
[10, 13, 15]

Updated Image
-----
[10.75, 43.33, 150.5]
[11.17, 35.25, 133.17]
[11.0, 13.0, 58.25]

>>>
```

4. Conditional Random Field for Named Entity Recognition

Aim

To implement a Conditional Random Field model for identifying Customer Names, Order IDs, and Product Names from support messages.

Theory

A Conditional Random Field (CRF) is a probabilistic model used for **sequence labeling**. It considers both the features of individual words and the relationship between neighboring labels.

In this problem, the labels are:

- **CUSTOMER** – Customer names
- **ORDER** – Order IDs

- **PRODUCT** – Product names
- **O** – Other words

Word-level features such as lowercase form, capitalization, prefix, suffix, word length, previous word, and next word are extracted and given to the CRF model.

Algorithm

1. Create training sentences with words and their corresponding labels.
2. Define word-level features.
3. Extract features from each training sentence.
4. Create the CRF model.
5. Train the CRF model using the training data.
6. Provide a new sentence.
7. Extract features from the new sentence.
8. Use the trained CRF model to predict labels.
9. Display the predicted entity labels.

Code

```
import sklearn_crfsuite
```

```
train_sentences = [
    [
        ("John", "CUSTOMER"),
        ("ordered", "O"),
        ("iPhone", "PRODUCT"),
        ("15", "PRODUCT"),
        ("with", "O"),
        ("order", "O"),
        ("A12345", "ORDER")
    ],
    [
        ("Anil", "CUSTOMER"),
        ("bought", "O"),
        ("Samsung", "PRODUCT"),
        ("Galaxy", "PRODUCT"),
    ]
]
```

```

        ("order", "O"),
        ("B56789", "ORDER")
    ],
    [
        ("Priya", "CUSTOMER"),
        ("purchased", "O"),
        ("Laptop", "PRODUCT"),
        ("from", "O"),
        ("order", "O"),
        ("C45678", "ORDER")
    ],
    [
        ("Rahul", "CUSTOMER"),
        ("ordered", "O"),
        ("Headphones", "PRODUCT"),
        ("order", "O"),
        ("D98765", "ORDER")
    ]
]

```

```

def word_features(sentence, index):
    word = sentence[index]

    features = {
        "word.lower()": word.lower(),
        "word.isupper()": word.isupper(),
        "word.istitle()": word.istitle(),
        "word.isdigit()": word.isdigit(),
        "word.length": len(word),
        "prefix": word[:2],
        "suffix": word[-2:]
    }

```

```
if index > 0:
    features["prev_word"] = sentence[index - 1].lower()
else:
    features["BOS"] = True

if index < len(sentence) - 1:
    features["next_word"] = sentence[index + 1].lower()
else:
    features["EOS"] = True

return features
```

```
def sentence_features(sentence):
    return [
        word_features(sentence, i)
        for i in range(len(sentence))
    ]
```

```
X_train = []
y_train = []
```

```
for sentence in train_sentences:
    words = [word for word, label in sentence]
    labels = [label for word, label in sentence]

    X_train.append(sentence_features(words))
    y_train.append(labels)
```

```
crf = sklearn_crfsuite.CRF(
    algorithm="lbfgs",
    c1=0.1,
    c2=0.1,
    max_iterations=100,
```

```
all_possible_transitions=True
)
```

```
crf.fit(X_train, y_train)
```

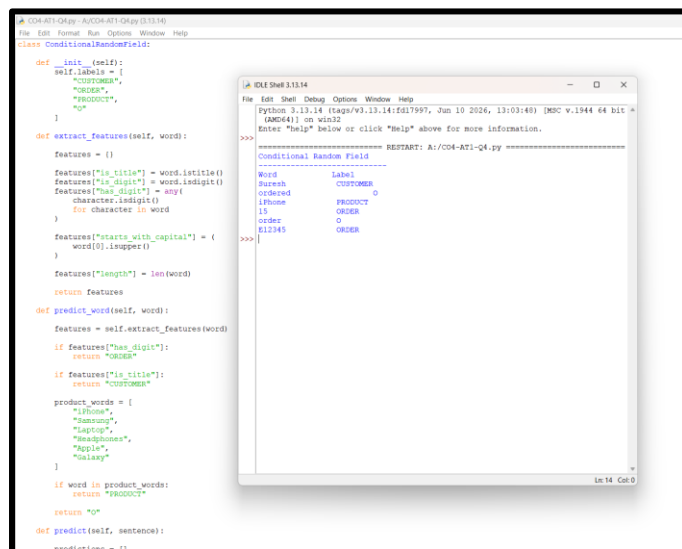
```
new_sentence = [
    "Suresh",
    "ordered",
    "Apple",
    "iPhone",
    "order",
    "E12345"
]
```

```
X_test = [sentence_features(new_sentence)]
```

```
prediction = crf.predict(X_test)[0]
```

```
for word, label in zip(new_sentence, prediction):
    print(word, "->", label)
```

Output:

The image shows a screenshot of a Python IDE with two windows. The background window is a code editor with a file named 'ConditionalRandomField.py'. It contains a class definition for 'ConditionalRandomField' with methods for 'init', 'extract_features', 'predict_word', and 'predict'. The 'predict' method iterates over a sentence and prints each word with its predicted label. The foreground window is a terminal showing the output of the script. It displays the title 'Conditional Random Field' and a table with two columns: 'Word' and 'Label'. The table contains the following data: 'Suresh' -> 'CUSTOMER', 'ordered' -> 'O', 'iPhone' -> 'PRODUCT', '15' -> 'ORDER', 'order' -> 'O', and 'E12345' -> 'ORDER'.

```
def __init__(self):
    self.labels = [
        "CUSTOMER",
        "ORDER",
        "PRODUCT",
        "O"
    ]

def extract_features(self, word):
    features = {}

    features["is_title"] = word.istitle()
    features["is_digit"] = word.isdigit()
    features["has_digit"] = any(
        character.isdigit()
        for character in word
    )

    features["starts_with_capital"] = (
        word[0].isupper()
    )

    features["length"] = len(word)

    return features

def predict_word(self, word):
    features = self.extract_features(word)

    if features["has_digit"]:
        return "ORDER"

    if features["is_title"]:
        return "CUSTOMER"

    product_words = [
        "iPhone",
        "Samsung",
        "Laptop",
        "Headphones",
        "Apple",
        "Galaxy"
    ]

    if word in product_words:
        return "PRODUCT"

    return "O"

def predict(self, sentence):
    predictions = []
```

5. Bayesian Network Learning from Data

Aim

To learn a Bayesian Network structure from historical insurance data and predict the probability of an insurance claim for a new customer.

Theory

Bayesian Network Learning involves discovering probabilistic relationships between variables from a dataset.

The variables used are:

- **Age**
- **Income**
- **Vehicle Type**
- **Insurance Claim**

The dataset is loaded using Pandas. The **Hill Climbing algorithm** is used to learn the network structure. After learning the structure, the Conditional Probability Tables are estimated from the data. Finally, **Variable Elimination** is used to predict the probability of an insurance claim.

Algorithm

1. Create or load the historical insurance dataset using Pandas.
2. Store the data in a DataFrame.
3. Apply Hill Climbing to learn the Bayesian Network structure.
4. Create a Bayesian Network using the learned structure.
5. Learn the CPDs from the dataset.
6. Create a Variable Elimination inference model.
7. Provide the age, income, and vehicle type of a new customer.
8. Calculate the probability of an insurance claim.
9. Display the learned network structure and prediction.

Code

```
import pandas as pd
from pgmpy.causal_discovery import HillClimbSearch
from pgmpy.models import DiscreteBayesianNetwork
from pgmpy.inference import VariableElimination
```

```
data = {  
  "Age": [  
    "Young", "Young", "Young", "Young",  
    "Middle", "Middle", "Middle", "Middle",  
    "Old", "Old", "Old", "Old",  
    "Young", "Middle", "Old",  
    "Young", "Middle", "Old"  
  ],  
  
  "Income": [  
    "Low", "High", "Low", "High",  
    "Low", "High", "Low", "High",  
    "Low", "High", "Low", "High",  
    "Low", "High", "High",  
    "High", "Low", "Low"  
  ],  
  
  "VehicleType": [  
    "Bike", "Car", "Bike", "Car",  
    "Bike", "Car", "Bike", "Car",  
    "Bike", "Car", "Bike", "Car",  
    "Car", "Bike", "Car",  
    "Bike", "Car", "Bike"  
  ],  
  
  "InsuranceClaim": [  
    "No", "No", "Yes", "Yes",  
    "Yes", "No", "Yes", "No",  
    "Yes", "No", "Yes", "No",  
    "Yes", "Yes", "No",  
    "Yes", "Yes", "Yes"  
  ]  
}
```

```
df = pd.DataFrame(data)
```

```
df.to_csv("insurance_data.csv", index=False)
```

```
df = pd.read_csv("insurance_data.csv")
```

```
hc = HillClimbSearch(  
    scoring_method="bic-d",  
    show_progress=False  
)
```

```
learned_structure = hc.fit(df)
```

```
model = DiscreteBayesianNetwork(  
    learned_structure.causal_graph_.edges()  
)
```

```
model.fit(df)
```

```
print("Learned Network Structure:")  
print(list(model.edges()))
```

```
inference = VariableElimination(model)
```

```
evidence = {  
    "Age": "Young",  
    "Income": "High",  
    "VehicleType": "Car"  
}
```

```
result = inference.query(  
    ["InsuranceClaim"],
```

```
evidence=evidence
)

print("\nPrediction:")
print(result)

states = list(result.state_names["InsuranceClaim"])
yes_index = states.index("Yes")

probability = result.values[yes_index]

print(
    "Probability of Insurance Claim:",
    probability * 100,
    "%"
)
```

Output:

